

Atividade Aula 005: Realização de 7 Exercícios Não Consecutivos da Lista

1. Linguagens imperativas e arquitetura de von Neumann:

Contexto: Imagine um programa que precisa calcular o preço final de um produto após aplicar um desconto.

Decisão tomada: Em uma linguagem imperativa, o programa pode armazenar o preço em uma variável, alterar seu valor por meio de atribuições e executar as instruções em sequência. Por exemplo: $\text{preço} = 100$, $\text{desconto} = 20$ e, depois, $\text{preço} = \text{preço} - \text{desconto}$.

Esse modelo está relacionado à arquitetura de von Neumann porque o programa trabalha com dados armazenados na memória e executa instruções de forma sequencial, modificando o estado das variáveis.

Consequência esperada: Ao final da execução, a variável preço terá o valor 80. O exemplo mostra que uma linguagem imperativa descreve como o estado do programa deve ser alterado passo a passo.

5. Variável:

Uma variável é uma entidade que possui um nome associado a um espaço de armazenamento e cujo valor pode ser alterado durante a execução do programa.

Exemplo: $\text{idade} = 18$ e, posteriormente, $\text{idade} = 19$. Nesse caso, idade é uma variável porque seu valor pode ser modificado durante a execução.

A variável está relacionada aos atributos das variáveis, pois possui características como nome, tipo, endereço, valor e tempo de vida.

Conclusão: variável não é apenas um nome utilizado no programa, mas uma entidade que possui características próprias e pode armazenar valores que podem ser modificados durante a execução.

9. Tempos de vinculação:

Os tempos de vinculação indicam quando uma característica de uma entidade da linguagem é associada a ela. Essa associação pode acontecer durante o projeto da linguagem, compilação, carregamento ou execução.

Na primeira situação, considere uma constante como $\text{PI} = 3.14$. Seu valor é definido antecipadamente e não sofre alterações durante a execução.

Na segunda situação, considere uma variável como $x = \text{lerValorDoUsuario}()$. Nesse caso, o valor de x só será conhecido durante a execução, pois depende da informação fornecida pelo usuário.

A diferença observável é que, no primeiro caso, o valor pode ser determinado antecipadamente, enquanto, no segundo, ele depende de uma informação obtida durante a execução.

Conclusão: considerar os tempos de vinculação é importante porque eles determinam

quando uma informação pode ser conhecida e quais decisões podem ser tomadas antes ou durante a execução do programa.

11. Declaração Explícita/Implícita:

Contexto: Considere um programa que utiliza uma variável chamada total.

Na declaração explícita, o programador informa diretamente as características da variável, como em inteiro total. Dessa forma, fica claro que total é uma variável do tipo inteiro.

Na declaração implícita, algumas linguagens permitem que a variável seja criada sem uma declaração formal, como em total = 50. Nesse caso, a linguagem pode determinar o tipo a partir do valor atribuído.

Decisão tomada: Se o objetivo for deixar o programa mais claro e facilitar a identificação de erros de tipo, pode-se utilizar uma declaração explícita.

Consequência esperada: O compilador terá uma informação clara sobre o tipo da variável e poderá identificar determinados erros mais cedo.

Conclusão: a declaração explícita informa diretamente as características da variável, enquanto a declaração implícita permite que parte dessas características seja determinada pela própria linguagem.

19. Escopo:

O escopo determina a região do programa na qual um nome ou variável pode ser utilizado.

Na primeira situação, considere uma variável declarada dentro de um bloco. Ela pode ser utilizada dentro desse bloco porque pertence ao seu escopo.

Na segunda situação, se o programa tentar utilizar essa mesma variável depois do fim do bloco, pode ocorrer um erro, pois o nome não está disponível naquele contexto.

A diferença observável é que, quando o escopo é considerado, é possível determinar exatamente onde uma variável pode ser acessada. Quando ele é ignorado, pode-se tentar utilizar um nome em uma região onde sua declaração não é válida.

Conclusão: o escopo é fundamental para determinar a visibilidade das entidades do programa e evitar acessos indevidos ou conflitos entre nomes.

21. Ocultação de nomes:

Contexto: Considere uma variável global chamada x com valor 10 e, dentro de um bloco, outra variável chamada x com valor 20.

Quando o programa utiliza x dentro do bloco, a variável mais interna é considerada, ocultando temporariamente a variável externa.

Decisão tomada: Dentro do bloco, a referência ao nome x deve utilizar a declaração mais interna.

Consequência esperada: Se o programa executar uma instrução para mostrar o valor de x dentro do bloco, o resultado será 20, e não 10.

Conclusão: a ocultação de nomes ocorre quando uma declaração em um escopo mais interno utiliza o mesmo nome de uma declaração existente em um escopo externo. A declaração interna passa a ser a referência utilizada naquele contexto. Esse conceito está diretamente relacionado ao escopo, pois é a organização dos escopos que determina qual declaração será encontrada.

22. Blocos e ordem de declaração:

O erro conceitual seria considerar blocos e ordem de declaração como características isoladas, sem perceber que eles estão relacionados ao escopo e à visibilidade dos nomes.

Um bloco delimita uma região do programa na qual determinadas declarações são válidas. Além disso, a ordem das declarações pode influenciar se um nome já pode ser utilizado.

Por exemplo, em um bloco no qual a variável a é declarada antes de b, a variável b pode utilizar a em sua inicialização. Porém, se b tentar utilizar a antes de a ser declarado, isso pode causar um erro em linguagens que exigem que o nome seja declarado antes de seu uso.

O problema de ignorar esses conceitos é não perceber que eles ajudam a determinar quais nomes estão disponíveis e em que momento podem ser referenciados.

Conclusão: blocos e ordem de declaração estão relacionados ao escopo e à resolução de nomes. Eles ajudam a determinar a região em que uma declaração é válida e o momento a partir do qual um nome pode ser utilizado, contribuindo para a correta interpretação do programa.